Módulo SDK Niubiz Connection

SDK ACTUAL:

Descripcion	El **Módulo SDK Niubiz Connection** permite la Integración con la Financiera Niubiz de Autoatendido IM30. Este módulo está diseñado para la operativa Kiosko.
Version	1.0.0
Package Name	-
Funcionalidades	Echo Test, Inicialización, Venta, Anulación, Cierre de lote, Reportes, Duplicado y Operaciones.
Código Fuente	No es propiedad de Niubiz
Estado	COMPLETADO

MANUAL DE ESPECIFICACIONES TÉCNICAS PARA INTEGRACIÓN

Tabla de Control de Cambios

Versión	Fecha	Razones Generales de Cambios	Autores
1.0	09/05/2024	• Documento Original.	• Mireya Gonzales
1.1	27/05/2024	• Aclaraciones sobre implementación en Linux	• Mireya Gonzales
1.2	14/11/2024	• Inclusión de “Implementación de Función de Operaciones“	• Jaeyson Bravo
1.3	27/11/2024	• Inclusión sobre puntos referente a la conexión de Integración y la salida a internet. (5.2)	• Jaeyson Bravo

Tabla de Índice

- 1 Objetivo General
- 2 Reserva Legal
- 3 Alcance
- 4 Definiciones
- 5 Requisitos Obligatorios para el APP del Comercio

- 6 Reglas de Implementación

Objetivo General

Este documento tiene como objetivo proveer los lineamientos para integrar soluciones de caja desarrolladas en Android, Windows y Linux por proveedores de software para comercios afiliados con nuestra aplicación financiera del terminal IM30.

Reserva Legal

Este documento contiene información y material confidencial propiedad de Compañía Peruana de Medios de Pago S.A.C. No está permitida su reproducción total o parcial ni su uso con otras organizaciones para ningún propósito, excepto en el caso de tener autorización previa por escrito.

Alcance

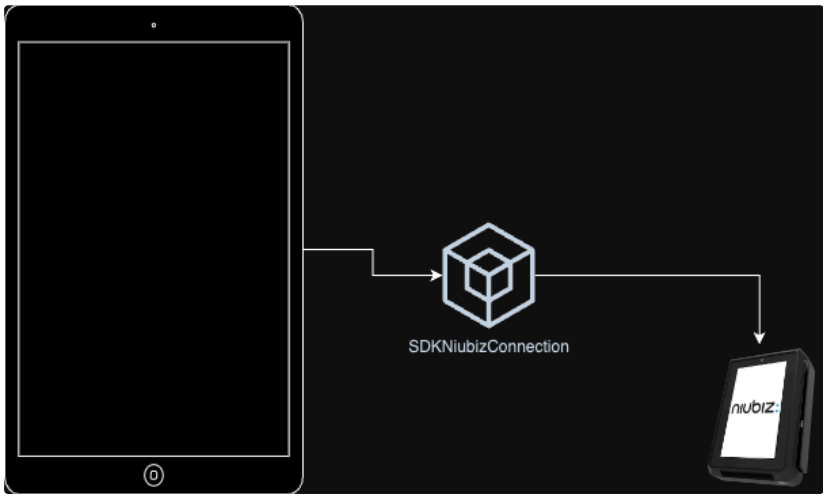
Este manual aplica a todos los comercios que necesiten integrar sus sistemas a nuestra solución de Autoatendido en la operativa Kiosko con la Financiera Niubiz en terminal IM30.

Definiciones

La aplicación Niubiz debe de estar instalada en el IM30 y el mismo debe de estar conectado al equipo dónde se desarrollará la integración. Cuándo tu aplicación envíe la instrucción de la operación, la aplicación Niubiz se encargará de ejecutar lo solicitado para luego mostrar el mensaje de éxito o error en la pantalla del IM30 y devolverá una respuesta a su aplicación.

4.1 Diagrama de Conexión

En el siguiente diagrama se visualiza que la comunicación hacia el IM30 siempre será usando el SDK, este proveerá al integrador una interfaz sencilla de implementar para hacer uso de las fuciones expuestas por la aplicación financiera de Niubiz.



Requisitos Obligatorios para el APP del Comercio

5.1 Consideraciones de desarrollo

- El área de Integraciones de Niubiz hará entrega de un equipo en modo debug para el desarrollo e integración con su aplicación y también el LINK de la carpeta compartida que contiene:

- Demo de ejemplo
- Manual de Especificaciones técnicas para Integración

- El equipo IM30 entregado por integraciones tendrá instalado el aplicativo Financiero de Niubiz y también un chip GPRS Niubiz. (El chip no aplica sí el IM30 va integrado a SK700).
- Sobre los SO Host (de la caja), se garantiza compatibilidad con:
  - Windows 10 64 bits versión 22H2
  - Linux Ubuntu
  - Android 10 en Totem SK700.
- **Actualmente la integración se encuentra disponible mediante lenguaje Java o Kotlin.**
- Es necesario tener el terminal inicializado para poder procesar ventas.
- Las siguientes funciones son de integración Obligatoria, si no están en la integración, el desarrollo presentado no se considerará efectivo.
  - Función de venta
  - Función de Inicialización
  - Función de Anulación (se recomienda colocar una contraseña de su control para la invocación).
  - Función de Duplicado (última transacción aprobada)
  - Función de Reportes

5.2 Consideraciones de Comunicación

- La comunicación de la integración es mediante el cable conectado a puerto USB.
- Opciones de conexión de red para la integración:
  - **USB:** el internet será compartido desde el sistema de autoatendido del cliente por este cable de conexión. Para esto, Integraciones en el kit compartido, brindará el manual correspondiente. En este punto, el comercio deberá de brindar permisos a la Whitelist indicada por Integraciones. No aplica por el momento para integraciones android distintas a IM30 con SK700
  - **RJ45:** La salida a internet será por medio de una conexión Ethernet. (Integraciones debe configurar esto en el terminal validando sí es DHCP o IP Fija). En este punto, el comercio deberá de brindar permisos a la Whitelist indicada por Integraciones.
  - **GPRS:** Este modo de conexión es mandatorio, todo terminal debe de salir con CHIP Niubiz, excepto IM30 integrado a SK700.

Reglas de Implementación

El código presentado en este manual es de tipo referencial y la intención es mostrar el código de una integración con nuestra aplicación Financiera Niubiz con un terminal IM30 Autoatendido.

6.1 Implementación de SDK en Android

En el archivo gradle del módulo hay que realizar las siguientes acciones:

- Importar el archivo “**SDKNiubizConnection.arr”** usando el archivo gradle de Android como se muestra a continuación:

```
implementation files('libs/ SDKNiubizConnection.arr')
```

Implementar librerías usadas como dependencias en SDKNiubizConnection

```
implementation("com.payneteasy:ber-tlv:1.0-11")
implementation("com.annimon:stream:1.2.1")
implementation("com.squareup.okio:okio:2.1.0")
```

Configurar NDK

```
android {
    …
    defaultConfig {
           …
           ndk {
                   abiFilters "armeabi-v7a", "armeabi", "x86"
           }

```

Seguidamente en tu archivo Manifest hay que agregar el atributo “extractNativeLibs” en true.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:name="com.startapps.demoniubiz.DemoNiubizApp"
        android:allowBackup="false"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:extractNativeLibs="true"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.DemoNiubiz"
        tools:targetApi="34">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>

```

Después del paso anterior hay que inicializar el SDK, para esto es necesario pasar un contexto de aplicación como en el siguiente fragmento de código.
Donde la variable “instance” es un contexto de aplicación.

```
SDKNiubizConnection.init(instance)
```

El siguiente fragmento de código solo aplica si estás en un dispositivo Android diferente al SK700:

Este fragmento de código se implementa en un activity:

```
private val customUsbReceiver: BroadcastReceiver = object : BroadcastReceiver() {
        override fun onReceive(arg0: Context, arg1: Intent) {
```

```
            when(arg1.action) {
                CommunicationConstants.ACTION_USB_PERMISSION -> {
                    Toast.makeText(
                        context,
                        "Permisos otorgados, listo comunicarse con IM30",
                        Toast.LENGTH_SHORT
                    ).show()
                }
                CommunicationConstants.ACTION_USB_ATTACHED -> {
                    Toast.makeText(
                        context,
                        "USB Conectado",
                        Toast.LENGTH_SHORT
                    ).show()

                    val devices = SDKNiubizConnection.getDevices()
                    if (devices.isNotEmpty()) {
                        SDKNiubizConnection.requestUserPermission(devices[0], context)
                    }
                }
                CommunicationConstants.ACTION_USB_DETACHED -> {
                    Toast.makeText(
                        context,
                        "USB desconectado",
                        Toast.LENGTH_SHORT
                    ).show()
                }
            }
        }
    }

override fun onCreate(savedInstanceState: Bundle?) {
        …
        val filter = IntentFilter()
        filter.addAction(CommunicationConstants.ACTION_USB_PERMISSION)
        filter.addAction(CommunicationConstants.ACTION_USB_DETACHED)
        filter.addAction(CommunicationConstants.ACTION_USB_ATTACHED)
        registerReceiver(customUsbReceiver, filter)
}

override fun onDestroy() {
        …
        unregisterReceiver(customUsbReceiver)
        super.onDestroy()
    }

```

Tener en consideración que en la implementación anterior la función SDKNiubizConnection.getDevices() retorna los dispositivos USB conectados, el integrador debe decidir cual es el que corresponde al IM30.

6.2 Implementación de SDK en Windows y Linux

Antes de realizar la integración a nivel de código, te recomendamos instalar los drivers para tu SO, la guía de como instalar está en 📲 Demo Windows / Linux dentro de la sección “Requisitos Técnicos“ según su Sistema Operativo.

Para iniciar la integración en tu aplicación debes importar el archivo **SDKNiubizConnection-release.jar** en el directorio que usas para colocar librerías externas. También es necesario importar las librerías **ber-tlv y jSerialComm**. Si la integración la haces desde java también es necesario incluir la librería **kotlin-stdlib-1.9.23.jar**. Los archivos mencionados se encuentran en el directorio para el SO Windows: **“01. WINDOWS-LINUX \ 00. WINDOWS \ 01. LIBRERÍA”**. Para el SO Linux: **“01. WINDOWS-LINUX \ 01. LINUX \ 01. LIBRERÍA”**



Después de esto debes configurar el puerto que se usará para la comunicación, para esto debes listar los puertos conectados y de estos seleccionar el que corresponde al IM30.

```
List<String> ports = SDKNiubizConnection.INSTANCE.getDevices();
// Listar puertos USB, uno de estos corresponde al IM30 conectado
for (String port : ports) {
     System.out.println("port=" + port);
}

//Debes ingresar el que corresponda al IM30
SDKNiubizConnection.INSTANCE.setCommDevice("COM3");

```

6.3 Puntos a tener en consideración

Si el SDK se usa desde Java es necesario usar la variable estática **INSTANCE**, en el caso de Kotlin no es necesario.

Java

```
SDKNiubizConnection.INSTANCE.sendEchoTest
```

Kotlin

```
SDKNiubizConnection.sendEchoTest
```

A partir de este punto el llamado a las funciones es independiente del dispositivo en el que te encuentres. Las llamadas a las funciones financieras son muy sencillas y solo se necesita de un “data class” donde la mayoría de parámetros son opcionales.

6.4 Implementación de Función de Echo Test

Esta función sirve para verificar la conexión entre IM30 y el host de Niubiz.

```
SDKNiubizConnection.sendEchoTest(EchoTestRequest()) {
      Log.e("MainActivity", "JSON=${Gson().toJson(it)}")
}
```

```
4
```

En el log se podrá observar en formato json el objeto que retorna el SDK, para el caso de la función sendEchoTest el objeto devuelto es de tipo EchoTestResponse.

A continuación se describe el contenido las variables del objeto EchoTestRequest:

- externalDeviceId: Identificador externo, solo se recibe y se retorna en la respuesta del SDK.
- cashierEmployeeNumber: Valor externo, solo se recibe y se retorna en la respuesta del SDK.

A continuación se describe el contenido las variables del objeto EchoTestResponse:

- isSuccess: Esta variable indica si la transacción fue existosa.
- dateTimeTransaction: Fecha y hora que respondió el host en la comunicación.
- externalDeviceId: Identificador externo, solo se recibe y se retorna en la respuesta del SDK.
- cashierEmployeeNumber: Valor externo, solo se recibe y se retorna en la respuesta del SDK.
- transactionResponseCode: Código de respuesta del host.
- transactionHostResponseText: Mensaje de respuesta del host.

6.4 Implementación de Función de Inicialización

Esta función sirve para iniciar el flujo de inicialización.

```
SDKNiubizConnection.sendInitialization(InitializationRequest()) {
      Log.e("MainActivity", "JSON=${Gson().toJson(it)}")
}

```

En el log se podrá observar en formato json el objeto que retorna el SDK, para el caso de la función sendInitialization el objeto devuelto es de tipo InitializationResponse.

A continuación se describe el contenido las variables del objeto InitializationRequest:

- externalDeviceId: Identificador externo, solo se recibe y se retorna en la respuesta del SDK.
- cashierEmployeeNumber: Valor externo, solo se recibe y se retorna en la respuesta del SDK.

A continuación se describe el contenido las variables del objeto InitializationResponse:

- isSuccess: Esta variable indica si la transacción fue existosa.
- dateTimeTransaction: Fecha y hora que respondió el host en la comunicación.
- externalDeviceId: Identificador externo, solo se recibe y se retorna en la respuesta del SDK.
- cashierEmployeeNumber: Valor externo, solo se recibe y se retorna en la respuesta del SDK.
- customerVoucher: Texto para imprimir de la incialización.

6.5 Implementación de Función de Venta

Esta función sirve para iniciar el flujo de venta.

```
SDKNiubizConnection.sendSale(SaleRequest()) {
      Log.e("MainActivity", "JSON=${Gson().toJson(it)}")
}

```

En el log se podrá observar en formato json el objeto que retorna el SDK, para el caso de la función sendSale el objeto devuelto es de tipo SaleResponse.

A continuación se describe el contenido las variables del objeto SaleRequest:

- totalAmount: Valor de la transacción para cobrar con tarjeta o billetera.
- externalDeviceId: Identificador externo, solo se recibe y se retorna en la respuesta del SDK.
- cashierEmployeeNumber: Valor externo, solo se recibe y se retorna en la respuesta del SDK.
- cashierNumber: Valor externo, solo se recibe y se retorna en la respuesta del SDK.
- posEntryModeCardOrQr: Sirve para seleccionar si el pago será por tarjeta o QR, si se envía null el IM30 mostrará una pantalla para seleccionar el método de pago.

A continuación se describe el contenido las variables del objeto SaleResponse:

- isSuccess: Esta variable indica si la transacción fue existosa.
- dateTimeTransaction: Fecha y hora en la que fue en la petición.
- externalDeviceId: Identificador externo, solo se recibe y se retorna en la respuesta del SDK.
- batchNumber: Lote transacción.
- transactionReference: Referencia de transacción.
- authorizedAmount: Monto autorizado.
- cardBrand: Marca de tarjeta
- maskedPan: Pan enmascarado
- transactionHostResponseText: Mensaje del host
- transactionResponseCode: Código de respuesta del host
- authorizationCode: Código de autorización
- installments: Número de cuotas
- deferredInstallments: Cuotas diferidas
- installmentAmount: Monto de cuota
- merchantVoucher: Texto para imprimir voucher de comercio
- customerVoucher: Texto para imprimir voucher de cliente
- transactionId: ID de transacción

6.6 Implementación de Función de Anulación

Esta función sirve para iniciar el flujo de anulación.

```
SDKNiubizConnection.sendVoidSale(VoidSaleRequest()) {
      Log.e("MainActivity", "JSON=${Gson().toJson(it)}")
}

```

En el log se podrá observar en formato json el objeto que retorna el SDK, para el caso de la función sendSale el objeto devuelto es de tipo VoidSaleResponse.

A continuación se describe el contenido las variables del objeto VoidSaleRequest:

- transactionReference: Referencia de transacción a anular, si no se envía el pinpad lo solicitará.
- externalDeviceId: Identificador externo, solo se recibe y se retorna en la respuesta del SDK.
- cashierEmployeeNumber: Valor externo, solo se recibe y se retorna en la respuesta del SDK.
- cashierNumber: Valor externo, solo se recibe y se retorna en la respuesta del SDK.

A continuación se describe el contenido las variables del objeto SaleVoidResponse:

- isSuccess: Esta variable indica si la transacción fue existosa.
- dateTimeTransaction: Fecha y hora en la que fue la petición.
- externalDeviceId: Identificador externo, solo se recibe y se retorna en la respuesta del SDK.
- batchNumber: Lote transacción.
- transactionReference: Referencia de transacción.
- transactionHostResponseText: Mensaje del host
- transactionResponseCode: Código de respuesta del host
- authorizationCode: Código de autorización
- merchantVoucher: Texto para imprimir voucher de comercio
- customerVoucher: Texto para imprimir voucher de cliente
- transactionId: ID de transacción

6.7 Implementación de Función de Cierre de Lote

Esta función sirve para iniciar la función de cierre de lote.

```
SDKNiubizConnection. sendBatchClosing(BatchClosingRequest()) {
      Log.e("MainActivity", "JSON=${Gson().toJson(it)}")
}

```

En el log se podrá observar en formato json el objeto que retorna el SDK, para el caso de la función sendBatchClosing el objeto devuelto es de tipo BatchClosingResponse.

A continuación se describe el contenido las variables del objeto BatchClosingRequest:

- transactionReference: Referencia de transacción a anular, si no se envía el pinpad lo solicitará.
- externalDeviceId: Identificador externo, solo se recibe y se retorna en la respuesta del SDK.
- cashierEmployeeNumber: Valor externo, solo se recibe y se retorna en la respuesta del SDK.
- cashierNumber: Valor externo, solo se recibe y se retorna en la respuesta del SDK.

A continuación se describe el contenido las variables del objeto BatchClosingResponse:

- isSuccess: Esta variable indica si la transacción fue exitosa.
- dateTimeTransaction: Fecha y hora en la que fue la petición.
- externalDeviceId: Identificador externo, solo se recibe y se retorna en la respuesta del SDK.
- cashierEmployeeNumber: Valor externo, solo se recibe y se retorna en la respuesta del SDK.
- transactionHostResponseText: Mensaje del host
- transactionResponseCode: Código de respuesta del host
- merchantVoucher: Texto para imprimir voucher de comercio

6.8 Implementación de Función de Reportes

Esta función sirve para iniciar la función de reportes.

```
SDKNiubizConnection. sendReport (ReportRequest ()) {
```

```
      Log.e("MainActivity", "JSON=${Gson().toJson(it)}")
}

```

En el log se podrá observar en formato json el objeto que retorna el SDK, para el caso de la función sendReport el objeto devuelto es de tipo ReportResponse.

- A continuación se describe el contenido las variables del objeto BatchClosingRequest:
- externalDeviceId: Identificador externo, solo se recibe y se retorna en la respuesta del SDK.
- cashierEmployeeNumber: Valor externo, solo se recibe y se retorna en la respuesta del SDK.

A continuación se describe el contenido las variables del objeto ReportResponse:

- isSuccess: Esta variable indica si la transacción fue existosa.
- dateTimeTransaction: Fecha y hora en la que fue la petición.
- externalDeviceId: Identificador externo, solo se recibe y se retorna en la respuesta del SDK.
- cashierEmployeeNumber: Valor externo, solo se recibe y se retorna en la respuesta del SDK.
- merchantVoucher: Texto para imprimir voucher de comercio

6.9 Implementación de Función de Duplicado

Esta función sirve para iniciar la función de duplicado.

```
SDKNiubizConnection. sendDuplicate (DuplicateRequest ()) {
      Log.e("MainActivity", "JSON=${Gson().toJson(it)}")
}

```

En el log se podrá observar en formato json el objeto que retorna el SDK, para el caso de la función sendDuplicate el objeto devuelto es de tipo DuplicateResponse.

- A continuación se describe el contenido las variables del objeto DuplicateRequest:
- transactionReference: Referencia de transacción a imprimir duplicado.
- externalDeviceId: Identificador externo, solo se recibe y se retorna en la respuesta del SDK.
- cashierEmployeeNumber: Valor externo, solo se recibe y se retorna en la respuesta del SDK.

A continuación se describe el contenido las variables del objeto DuplicateResponse:

- isSuccess: Esta variable indica si la transacción fue existosa.
- transactionReference: Referencia de transacción a imprimir duplicado.
- merchantVoucher: Texto para imprimir voucher de comercio.
- customerVoucher: Texto para imprimir voucher de cliente.

6.10 Implementación de Función de Operaciones

Esta función sirve para iniciar la función de Operaciones.

```
SDKNiubizConnection.sendSale(
    SaleRequest(
```

```
        totalAmount = BigDecimal(808080),
        posEntryModeCardOrQr = null
    )
) {

}

```

En el fragmento de código fuente anterior se observa que se usa el objeto SaleRequest con valores específicos (utilizar esos valores). En este caso no hay objeto devuelto.